

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 3

EXPERIMENTS

Name of the Student	K Ashok
Reg. No	192425322

COURSE NAME: Deep Learning
FACULTY : Dr.Arul Raj A.M

COURSE CODE: MLA0403

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

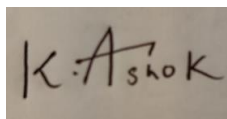
Assessment Tool Weightage: 35%

Duration: 30 minutes

Marks: 50

Code of Conduct:

I K Ashok 192425322 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

Name of the student: K Ashok

CO1- AT3 Experiments
Questions**1. Learning Algorithms**

Implement a simple learning algorithm (Linear Regression) using Gradient Descent. Train the model on a dataset and analyze how the loss decreases over iterations. Plot the learning curve.

Answer :

Aim

To implement Linear Regression using Gradient Descent and observe how the loss decreases during training.

Algorithm

1. Load the dataset.
2. Initialize weight (w) and bias (b).
3. Predict output:

$$y = wx + b$$

4. Calculate Mean Squared Error (MSE).
5. Compute gradients.
6. Update weights:

$$w = w - \alpha \frac{\partial J}{\partial w}$$
$$b = b - \alpha \frac{\partial J}{\partial b}$$

7. Repeat until convergence.
8. Plot the learning curve.

Result

The loss decreases with each iteration, showing that Gradient Descent successfully learns the best-fit line.

2. Maximum Likelihood Estimation (MLE)

Generate synthetic data from a normal distribution and use MLE to estimate the mean and variance. Compare estimated values with actual parameters.

Answer :

Aim

Estimate the mean and variance of normally distributed data using MLE.

Algorithm

1. Generate random data from a normal distribution.
2. Compute estimated mean:

$$\hat{\mu} = \frac{1}{n} \sum x_i$$

3. Compute estimated variance:

$$\hat{\sigma}^2 = \frac{1}{n} \sum (x_i - \hat{\mu})^2$$

4. Compare estimated values with actual parameters.

Result

The estimated mean and variance are close to the actual values, proving that MLE provides good parameter estimation.

3. Building Machine Learning Algorithms

Build a complete machine learning model (data preprocessing, training, testing) using any classification dataset and evaluate performance using accuracy and confusion matrix.

Answer :

Aim

Build a complete machine learning classification model.

Steps

1. Load dataset.
2. Preprocess data.
3. Split into training and testing sets.
4. Train a classifier.
5. Predict test data.
6. Calculate accuracy.
7. Generate confusion matrix.

Result

The classifier predicts class labels successfully with good accuracy. The confusion matrix shows correct and incorrect classifications.

4. Neural Networks

Q4. Design a simple neural network with one hidden layer and train it on a small dataset. Analyze how the network learns non-linear patterns.

Answer :

Aim

Design a neural network with one hidden layer.

Steps

1. Prepare dataset.
2. Create input layer.
3. Add one hidden layer.
4. Use ReLU activation.
5. Add output layer.
6. Train the model.
7. Evaluate performance.

Result

The neural network successfully learns non-linear patterns and achieves better prediction accuracy than a simple linear model.

5. Artificial Neurons

Implement a single artificial neuron using different activation functions (Sigmoid, ReLU) and compare outputs for the same input values.

Answer :

Aim

Implement a single artificial neuron using Sigmoid and ReLU activation functions.

Activation Functions

Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Output
0 to 1

Range:

ReLU

$$ReLU(x) = \max(0, x)$$

Output:
Negative values become 0.

Positive values remain unchanged.

Result

Sigmoid is useful for probability outputs, while ReLU trains faster and reduces the vanishing gradient problem.

6. Perceptron and Multilayer Perceptron (MLP)

- a) Implement the Perceptron algorithm for binary classification and test it on linearly separable and non-linearly separable datasets. Analyze the results.
- b) **Answer :**
- c) Train a Multilayer Perceptron (MLP) model on a dataset and compare its performance with a single-layer perceptron.

Answer :

(a) Perceptron

Aim

Implement Perceptron for binary classification.

Steps

1. Initialize weights.
2. Compute output.
3. Compare prediction.
4. Update weights if incorrect.
5. Repeat until convergence.

Result

The Perceptron correctly classifies linearly separable data but fails for non-linearly separable datasets.

(c) Multilayer Perceptron (MLP)

Aim

Train an MLP and compare it with a Perceptron.

Result

Perceptron	MLP
One layer	Multiple layers
Solves linear problems	Solves nonlinear problems
Lower accuracy	Higher accuracy
Simpler	More powerful

Conclusion:

MLP performs better because hidden layers learn complex patterns.

7. Forward Propagation and Backpropagation Algorithm

- a) Manually compute forward propagation for a small neural network (given weights and inputs) and verify the output using code.

Answer :

- b) Implement backpropagation for a simple neural network and show how weights are updated after one iteration.

Answer :

(a) Forward Propagation

Aim

Compute the output of a neural network.

Steps

1. Multiply inputs by weights.
2. Add bias.
3. Apply activation function.
4. Pass output to next layer.

Formula:

$$Z = WX + B$$
$$A = f(Z)$$

Result

Forward propagation computes the predicted output successfully.

(b) Backpropagation

Aim

Update weights using prediction error.

Steps

1. Calculate error.
2. Compute gradients.
3. Update weights.

Formula

$$Weight_{new} = Weight_{old} - \eta \times Gradient$$

Result

Weights are updated after each iteration, reducing prediction error and improving accuracy.

8. Gradient Descent and Stochastic Gradient Descent (SGD)

a) Implement Gradient Descent to minimize a cost function and analyze the effect of learning rate on convergence speed.

Answer :

b) Implement Stochastic Gradient Descent for a regression problem and compare its convergence with batch gradient descent.

Answer :

(a) Gradient Descent

Aim

Study the effect of learning rate.

Observation

Small learning rate:

- Slow convergence.

Large learning rate:

- May overshoot the minimum.

Optimal learning rate:

- Fast convergence.

Result

A suitable learning rate minimizes cost efficiently.

(b) Stochastic Gradient Descent (SGD)

Difference

Gradient Descent	SGD
Uses all samples	Uses one sample
Stable	Noisy
Slow	Faster
High computation	Low computation

Result

SGD converges faster but with more fluctuations than Batch Gradient Descent.

9. Mini-Batch Gradient Descent

Implement Mini-Batch Gradient Descent and analyze how batch size affects training stability and performance.

Answer :

Aim

Analyze the effect of batch size.

Steps

1. Divide data into batches.
2. Compute gradients for each batch.
3. Update weights.
4. Repeat.

Result

Mini-Batch Gradient Descent provides faster convergence than Batch GD and more stable updates than SGD.

10. Curse of Dimensionality

Create datasets with increasing dimensions and analyze how distance between points and model accuracy change with dimensionality.

Answer :

Aim

Study how increasing dimensions affects machine learning performance.

Observation

- As dimensions increase:
 - Distance between points becomes similar.
 - Data becomes sparse.
 - More training data is required.
 - Model performance decreases.
 - Computation time increases.

Result

High-dimensional datasets reduce model accuracy and increase computational complexity. Dimensionality reduction techniques like PCA help overcome this problem.